

Computação Distribuída com MPI: Multiplicação de Matrizes e Método de Monte Carlo

Relatório de Atividade de Laboratório

Prof. Alcides / Prof. Mário

Campo	Preenchimento
Nome completo	Pedro Roberto Fernandes Noronha
RA	10443434
Turma	Turma 201815117.000.06N
Disciplina	Computação distribuída
Data de entrega	20/09/2026

1. Introdução e Fundamentação Teórica

Esta atividade tem por objetivo implementar e avaliar duas aplicações paralelas sobre um cluster distribuído simulado por containers Docker, utilizando o padrão MPI (Message Passing Interface) através da biblioteca mpi4py.

1.1 Modelo de execução do MPI

MPI cria cópias independentes do mesmo programa e roda simultaneamente, sem hierarquia embutida e com trechos de memória isolados inicialmente. As cópias são diferenciadas pelo identificador rank, que vai de 0 a n-1. Permitindo alternar trechos executados por apenas um processo ou delimitados por faixa de trabalho.

1.2 Comunicação coletiva

As três operações coletivas utilizadas nesta atividade foram:

- `comm.bcast` — um processo (root) transmite um dado a todos os demais. Utilizada no Exercício 1 para distribuir as matrizes A e B.
- `comm.gather` — todos os processos enviam seus resultados parciais ao root, que os recebe como uma lista ordenada por rank. Utilizada no Exercício 1 para reunir as fatias da matriz C.
- `comm.reduce` — todos enviam um valor ao root, que já os combina por uma operação (aqui, `MPI.SUM`). Utilizada no Exercício 2 para somar as contagens locais.

O exercício 2 precisa apenas concluir uma operação, não são necessários os valores originais, `gather` seria viável mas devolveria uma lista que demandaria mais cálculo.

1.3 Partição de dados (data partitioning)

O exercício 1 particiona os dados através de faixas regulares de linhas, cada processo tem um n de rank diferente, o que gera uma divisão sem sobreposição

```
23  linhas_por_processo = N // size
24  linha_ini = rank * linhas_por_processo
25  linha_fim = linha_ini + linhas_por_processo
26
```

O exercício 2 particiona os dados através de uma divisão simples, já que os pontos não tem endereços. Um ponto importante, é necessário assumir que `size` é divisível por `N`, como todos os testes solicitados o são, foi mantido assim.

```
n_local = N // size
```

1.4 Método de Monte Carlo

Um quadrado de lado 1 possui área igual a 1, e o quarto de círculo de raio 1 nele inscrito possui área $\pi/4$. Sorteando-se pontos uniformemente distribuídos no quadrado, a fração que recai no interior do quarto de círculo tende à razão entre as duas áreas, ou seja, a $\pi/4$. Segue que π é aproximadamente quatro vezes essa fração. O teste de pertencimento é dado por $x^2 + y^2 \leq 1$, aplicação direta do teorema de Pitágoras comparando a distância do ponto à origem com o raio.

A precisão do método cresce com a raiz quadrada do número de amostras, de modo que reduzir o erro pela metade exige quadruplicar a quantidade de pontos. As execuções realizadas confirmam esse comportamento: o erro relativo foi de 1,20% com mil pontos, 0,05% com cem mil e 0,018% com dez milhões.

2. Metodologia e Ambiente de Execução

2.1 Ambiente computacional

Item	Configuração
Plataforma	GitHub Codespaces
Núcleos disponíveis (nproc)	2
Sistema base dos containers	Ubuntu 22.04
Implementação MPI	OpenMPI
Versão do Python	Python 3.14.2
Versão do mpi4py	3.1.3
Versão do OpenMPI	mpirun (Open MPI) 4.1.2

2.2 Topologia do cluster

O cluster foi composto por quatro containers Docker conectados por uma rede bridge dedicada: um nó master e três nós workers. A comunicação entre os nós ocorre por SSH sem senha, com chave pública compartilhada.

O arquivo hosts lista os nós com 2 de slot, visando o paralelismo correto de acordo com os propósitos da atividade, foi utilizado o `--map-by node`.

2.3 Versões implementadas

Para cada problema foram implementadas três versões, de modo a permitir a comparação de desempenho:

- Sequencial — execução em um único processo, sem paralelismo.
- Multithreaded — quatro threads em uma única máquina (apenas Exercício 1).
- Distribuída com MPI — quatro processos, um por container.

3. Resultados e Evidências de Execução

3.1 Tabela de desempenho — Exercício 1

Tempos de execução da multiplicação de matrizes, em milissegundos:

Dimensão (N)	Sequencial (ms)	Multithreaded (ms)	Distribuído MPI (ms)
N = 300	7.116,65	2.827,09	4.699,81
N = 600	56.997,63	26.024,41	36.887,24
N = 1000	281.819,93	132.778,24	175.803,21

3.2 Tabela de desempenho — Exercício 2

Estimativa de π pelo método de Monte Carlo com N = 10.000.000 pontos:

Versão	π estimado	Erro absoluto	Tempo (ms)
Sequencial	3,141538	0,000055	4.913,11
Distribuída MPI	3,141754	0,000161	3.012,95

Valor de referência: $\pi = 3,141593$.

3.3 Capturas de tela obrigatórias

— Figura 1 — subida dos containers (docker compose up -d e docker ps)

```
@pedroroberto12 →/workspaces/Atividade-Lab---Computa-o-Distribu-da-com-MPI-Multiplica-o-de-Matrizes-e-M-todo-de-Monte-Carlo (main) $ docker ps --format "table {{.Names}}\t{{.Status}}"
NAMES      STATUS
master     Up About a minute
worker3     Up About a minute
worker2     Up About a minute
worker1     Up About a minute
```

— Figura 2 — SSH configurado e teste mpirun --map-by node -np 4 hostname

```
worker1 Up About a minute
@pedroroberto12 →/workspaces/Atividade-Lab---Computa-o-Distribu-da-com-MPI-Multiplica-o-de-Matrizes-e-M-todo-de-Monte-Carlo (main) $ sudo iptables-legacy -P FORWARD ACCEPT

./setup_ssh.sh
docker compose exec master su - mpiuser -c "mpirun --hostfile hosts --map-by node -np 4 hostname"
1/3 Gerando a chave no master...
Generating public/private rsa key pair.
Your identification has been saved in /home/mpiuser/.ssh/id_rsa
Your public key has been saved in /home/mpiuser/.ssh/id_rsa.pub
The key's fingerprint is:
SHA256:Qn1Aos1878SNWksWgZ9MLEdGD+cjghft8ERDrdR7Vg root@master
The key's randomart image is:
+----[RSA 2048]-----+
| ..+... |
| o o+ + E |
|..o...o+ + |
|+o o* o. . |
|..o +XS |
|..+..o B.. |
|o..o= + + |
|+o..o= + |
|oo..+o |
+----[SHA256]-----+
2/3 Autorizando a chave em cada no...
-> master
-> worker1
-> worker2
-> worker3
3/3 Criando o hostfile no master...

SSH configurado. Teste com:
docker compose exec master su - mpiuser -c "mpirun --hostfile hosts --map-by node -np 4 hostname"
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
master
worker2
worker1
worker3
```

— Figura 3 — execução do Exercício 1 no master, com os tempos

```
@peedroroberto12 →/workspaces/Atividade-Lab---Computa-o-Distribu-da-com-MPI-Multiplica-o-de-Matrizes-e-M-todo-de-Monte-Carlo (main) $ ./run.sh exercicio1.py 300
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
N = 300 | C acumulado: 6732012.8222 | tempo: 5573.48 ms
@peedroroberto12 →/workspaces/Atividade-Lab---Computa-o-Distribu-da-com-MPI-Multiplica-o-de-Matrizes-e-M-todo-de-Monte-Carlo (main) $ ./run.sh exercicio1.py 600
[master:00117] plm:rs: Warning: setpgid(124,124) failed in parent with errno=Permission denied(13)
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
N = 600 | C acumulado: 54030305.3860 | tempo: 39109.16 ms
```

— Figura 4 — execução do Exercício 2, com a estimativa de π e o tempo

```
@peedroroberto12 →/workspaces/Atividade-Lab---Computa-o-Distribu-da-com-MPI-Multiplica-o-de-Matrizes-e-M-todo-de-Monte-Carlo (main) $ ./run.sh exercicio2.py 1000000
Warning: Permanently added 'worker1' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker3' (ED25519) to the list of known hosts.
Warning: Permanently added 'worker2' (ED25519) to the list of known hosts.
PI ≈ 3.141754 | tempo: 3012.95 ms
@peedroroberto12 →/workspaces/Atividade-Lab---Computa-o-Distribu-da-com-MPI-Multiplica-o-de-Matrizes-e-M-todo-de-Monte-Carlo (main) $ python3 montecarlo_seq.py 1000000
PI aproximado: 3.141538
Tempo sequencial: 4913.11 ms
@peedroroberto12 →/workspaces/Atividade-Lab---Computa-o-Distribu-da-com-MPI-Multiplica-o-de-Matrizes-e-M-todo-de-Monte-Carlo (main) $
```

— Figura 5 — tabela consolidada gerada pelo benchmark.sh

Exercicio 1 - Multiplicacao de matrizes tempos em ms			
N	Sequencial	Threads	MPI (4 nos)
300	7116.65	2827.09	4699.81
600	56997.63	26024.41	36887.24
1000	281819.93	132778.24	175803.21
Gerado em 19/09/2026 17:12 nproc = 2			

4. Análise de Desempenho e Discussão

4.1 Speedup obtido

Caso	Sequencial (ms)	MPI (ms)	Speedup
Matrizes N = 300	7.116,65	4.699,81	1,51×
Matrizes N = 600	56.997,63	36.887,24	1,55×
Matrizes N = 1000	281.819,93	175.803,21	1,60×
Monte Carlo N = 10M	4.913,11	3.012,95	1,63×

4.2 Verificação da complexidade $O(N^3)$

A passagem de $N = 300$ para $N = 600$ elevou o tempo sequencial de 7.116,65 ms para 56.997,63 ms, um fator de 8,01×. O valor corresponde ao previsto pela complexidade do algoritmo.

4.3 Efeito do limite de núcleos

A máquina virtual do codespace conta com apenas 2 núcleos, isso estabelece um limite teórico de 2x de speedup, os valores obtidos chegam a 80% do limite aproximadamente.

4.4 Overhead de comunicação

O crescimento na ordem de $N (1,51 \times \rightarrow 1,55 \times \rightarrow 1,60 \times)$ indica que o overhead de comunicação tende a se diluir conforme o volume de cálculo aumenta, entretanto em operações pequenas o processamento gasto com comunicação é muito significativo

4.5 Análise crítica da coluna multithreaded

A versão com threads apresentou tempos inferiores aos da versão sequencial — 2,12× mais rápida em $N = 1000$. O valor excede o teto de 2× imposto pelos dois núcleos disponíveis, o que indica que o ganho não decorre de paralelismo.

Um experimento controlado foi conduzido para isolar as causas. Mantendo o mesmo kernel de cálculo e variando apenas o escopo das variáveis e o número de threads, obteve-se:

Configuração	Tempo relativo	Interpretação
Laço no nível do módulo (variáveis globais)	referência	—
Laço dentro de função, 1 thread	2,34× mais rápido	efeito de escopo
Laço dentro de função, 4 threads	0,94×	efeito do GIL

A diferença não está no paralelismo e sim em onde os laços foram escritos: no sequencial.py acessam variáveis globais, no threads.py estão dentro de uma função e acessam locais. No CPython global exige busca em dicionário a cada leitura, e num laço de 27 milhões de iterações isso responde pela maior parte do ganho.

O experimento confirma: fixando os laços na função e indo de 1 para 4 threads, o tempo ficou em 0,94×, levemente pior. O paralelismo real é nulo por causa do GIL. O 2,12× da tabela é artefato da comparação e ainda ultrapassa o teto de 2× dos dois núcleos.

4.6 Quando o cluster distribuído se justifica

Em Python threads não resolvem o trabalho de processamento, o GIL serializa a execução, threads ajudam apenas em tarefas de entrada e saída, onde o GIL é liberado durante a espera.

Em processos separados, isso não acontece. Em contrapartida todo dado precisa ser transmitido o que gera overhead, o que define se o cluster se justifica então e o volume de processamento sobrepondo o impacto do transporte de dados, abaixo disso o overhead domina.

Os dois casos representam formas opostas de comunicação, no primeiro exercício as matrizes são enviadas completas e depois serializadas, no exercício 2 apenas 4 inteiros trafegam no reduce.

Seria de esperar, então, que o Monte Carlo escalasse bem melhor. Os resultados não mostram isso. A explicação é o teto de 2 núcleos do ambiente, que os dois casos já quase atingiram, a 80% e 81% do máximo possível. Sem margem, a vantagem da comunicação leve não teve como aparecer.

A conclusão é que este experimento não conseguiu distinguir os dois regimes. A diferença se manifestaria num cluster com mais núcleos físicos, onde o Monte Carlo continuaria escalando e a multiplicação de matrizes empacaria no custo de transmitir as matrizes.

5. Dificuldades Encontradas e Soluções

Os problemas abaixo foram efetivamente enfrentados durante a atividade.

5.1 Tráfego bloqueado entre containers

O comando `mpirun` gerava travamento, foi necessário resolver a conexão com o comando: `sudo iptables-legacy -F FORWARD`.

5.2 Sincronização do código com os nós

Editar os arquivos não altera a copia existente nos contêineres, para subir as alterações e necessário executar `Docker cp`, o `run.sh` faz os 4 `docker cp` de uma vez, automatizando o processo.

6. Uso de Inteligência Artificial

6.1 Ferramentas consultadas

Claude, modelo Opus 5, esforço Alto.

6.2 Etapas em que houve auxílio

A ia foi utilizado como ferramenta de consulta na escrita do código principalmente, auxiliando com dilemas lógicos, consulta de sintaxe e funções específicas, formatação de strings, e elaboração de scripts para testes com boa visualização, além de ajudar com a subida dos contêineres.

Já o relatório apenas foi estruturado o docx de forma a ter um esqueleto organizado, com tabelas e espaços demarcados para prints, as análises críticas forma preenchidas por mim apenas com auxílio em reflexões teóricas.

6.3 Principais prompts utilizados

Seguem dois prompts relevantes para entender o uso da ia no projeto, copia e cola na tentativa de resolver um erro, e ajudana elaboração de scripts para teste

DOCKER-USER 0 -- 0.0.0.0/0 0.0.0.0/0

@peedroroberto12 → /workspaces/Atividade-Lab---Computa-o-Distribu-da-com-MPI-Multiplica-o-de-Matrizes-e-M-todo-de-Monte-Carlo (main) \$ sudo iptables-legacy -L FORWARD -n | head -3

Chain FORWARD (policy DROP)

target prot opt source destination

DOCKER-USER 0 -- 0.0.0.0/0 0.0.0.0/0

@peedroroberto12 → /workspaces/Atividade-Lab---Computa-o-Distribu-da-com-MPI-Multiplica-o-de-Matrizes-e-M-todo-de-Monte-Carlo (main) \$

modifica aql script de run pra rodar todos os testes nas 3 quantidades e nos 3 modelos e me dar uma saída compreensível tem como?

6.4 Análise crítica

A ferramenta muitas vezes erra nas explicações teóricas, quando é necessário investigar a causa de algum resultado por exemplo, as vezes sugerindo explicações mirabolantes por não possuir o contexto completo, como o número minúsculo de núcleos da máquina do codespace.

Em relação a escrita de código, as soluções sugeridas pelo claude costumam ser muito boas, mas precisam de atenção para não serem extremamente ilegíveis e complexas, em relação aos scripts tudo funcionou e automatizou uma tarefa manual que levaria mais tempo que o necessário.

7. Conclusão

O aprendizado teórico mais duradouro e valioso da atividade foi a diferença entre os efeitos da paralelização em situações de grande ou pequeno processamento de dados, além dos problemas da comunicação entre processos. Do ponto de vista prático aprendi melhor a como subir e operar contêineres, formular scripts em python, e interpretar resultados.

Do ponto de vista metodológico, ficou claro que um resultado quando fora de contexto pode apresentar uma conclusão equivocada, sendo necessário isolar algumas variáveis para obter uma resposta mais satisfatória.

A versão com threads aparentava ganho de paralelismo, mas o experimento controlado revelou que a diferença vinha do escopo das variáveis, e que o paralelismo real era nulo por causa do GIL. Verificar antes de concluir se mostrou mais útil do que aceitar o número medido.

8. Referências

mpi4py — MPI for Python. Documentação oficial. Disponível em: <https://mpi4py.readthedocs.io/>

MPI Forum. MPI: A Message-Passing Interface Standard. Disponível em: <https://www.mpi-forum.org/docs/>

Open MPI. Documentação oficial. Disponível em: <https://www.open-mpi.org/doc/>

Python Software Foundation. Documentação do módulo threading. Disponível em: <https://docs.python.org/3/library/threading.html>

Apêndice — Código-fonte

Reproduzem-se a seguir, na íntegra, os arquivos utilizados na atividade: as cinco implementações avaliadas e os cinco arquivos de infraestrutura e automação do cluster.

A.1 exercicio1.py

Exercício 1 — multiplicação de matrizes distribuída (bcast + gather)

```
import random
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

import sys
N = int(sys.argv[1]) if len(sys.argv) > 1 else 300

if rank == 0:
    A = [[random.random() for _ in range(N)] for _ in range(N)]
    B = [[random.random() for _ in range(N)] for _ in range(N)]
else:
    A = None
    B = None

comm.Barrier()
t0 = MPI.Wtime()
A = comm.bcast(A, root=0)
B = comm.bcast(B, root=0)

linhas_por_processo = N // size
linha_ini = rank * linhas_por_processo
linha_fim = linha_ini + linhas_por_processo
```

```

bloco_local = []
for i in range(linha_ini, linha_fim):
    linha_C = []
    for j in range(N):
        soma = 0.0
        for k in range(N):
            soma += A[i][k] * B[k][j]
        linha_C.append(soma)
    bloco_local.append(linha_C)

blocos = comm.gather(bloco_local, root=0)
t_fim = MPI.Wtime()

if rank == 0:
    C = []
    for bloco in blocos:
        C.extend(bloco)
    soma_C = sum(sum(linha) for linha in C)
    print(f"N = {N} | C acumulado: {soma_C:.4f} | tempo: {(t_fim - t0) * 1000:.2f} ms")

```

A.2 exercicio2.py

Exercício 2 — estimativa de π por Monte Carlo (reduce)

```

import sys
from mpi4py import MPI

comm = MPI.COMM_WORLD
rank = comm.Get_rank()
size = comm.Get_size()

N = int(sys.argv[1]) if len(sys.argv) > 1 else 10_000_000
n_local = N // size

import random

dentro_local = 0
comm.Barrier()
t0 = MPI.Wtime()
for _ in range(n_local):
    x = random.random()
    y = random.random()
    if x * x + y * y <= 1.0:
        dentro_local += 1

dentro_total = comm.reduce(dentro_local, op=MPI.SUM, root=0)
t_fim = MPI.Wtime()

if rank == 0:
    pi = 4.0 * dentro_total / N
    print(f"PI ~ {pi:.6f} | tempo: {(t_fim - t0) * 1000:.2f} ms")

```

A.3 sequencial.py

Baseline sequencial da multiplicação de matrizes

```

import random
import time

import sys
N = int(sys.argv[1]) if len(sys.argv) > 1 else 300

A = [[random.random() for _ in range(N)] for _ in range(N)]
B = [[random.random() for _ in range(N)] for _ in range(N)]
C = [[0] * N for _ in range(N)]

inicio = time.time()

for i in range(N):

```

```

        for j in range(N):
            for k in range(N):
                C[i][j] += A[i][k] * B[k][j]

fim = time.time()

print(f"Tempo sequencial: {(fim - inicio) * 1000:.2f} ms")

```

A.4 threads.py

Baseline multithreaded da multiplicação de matrizes

```

import threading
import random
import time

import sys
N = int(sys.argv[1]) if len(sys.argv) > 1 else 300

THREADS = 4

A = [[random.random() for _ in range(N)] for _ in range(N)]
B = [[random.random() for _ in range(N)] for _ in range(N)]
C = [[0] * N for _ in range(N)]

def calcular(inicio, fim):
    for i in range(inicio, fim):
        for j in range(N):
            for k in range(N):
                C[i][j] += A[i][k] * B[k][j]

inicio = time.time()

threads = []
linhas_por_thread = N // THREADS

for t in range(THREADS):
    ini = t * linhas_por_thread
    fim_t = N if t == THREADS - 1 else (t + 1) * linhas_por_thread
    th = threading.Thread(target=calcular, args=(ini, fim_t))
    threads.append(th)
    th.start()

for th in threads:
    th.join()

fim = time.time()

print(f"Tempo com threads: {(fim - inicio) * 1000:.2f} ms")

```

A.5 montecarlo_seq.py

Baseline sequencial do método de Monte Carlo

```

import random
import time

N = 10_000_000
dentro = 0
inicio = time.time()

for _ in range(N):
    x = random.random()
    y = random.random()
    if x * x + y * y <= 1.0:
        dentro += 1

pi_estimado = 4.0 * dentro / N

```

```

fim = time.time()

print(f"PI aproximado: {pi_estimado:.6f}")
print(f"Tempo sequencial: {(fim- inicio) * 1000:.2f} ms")

```

A.6 Dockerfile

Imagem base dos nós do cluster

```

FROM ubuntu:22.04

ENV DEBIAN_FRONTEND=noninteractive

# Base identica a do Lab 01 do professor, com duas adicoes:
# python3 / python3-pip -> necessarios para o Lab 03
# python3-mpi4py -> o mpi4py ja compilado contra o OpenMPI.
# O Lab 03 manda instalar o mpi4py depois, com 8 comandos nos 4 containers.
# Instalando aqui, ele ja nasce pronto e sobrevive a qualquer rebuild.
RUN apt update && \
    apt install -y openmpi-bin libopenmpi-dev openssh-server sudo figlet net-tools iputils-ping \
        python3 python3-pip python3-mpi4py && \
    useradd -m -s /bin/bash mpiuser && \
    echo "mpiuser:mpi123" | chpasswd && \
    adduser mpiuser sudo && \
    mkdir /var/run/ssh

EXPOSE 22
CMD service ssh start && sleep infinity

```

A.7 docker-compose.yml

Definição dos quatro containers e da rede

```

services:
  master:
    build: .
    container_name: master
    hostname: master
    networks:
      - mpi_net

  worker1:
    build: .
    container_name: worker1
    hostname: worker1
    networks:
      - mpi_net

  worker2:
    build: .
    container_name: worker2
    hostname: worker2
    networks:
      - mpi_net

  worker3:
    build: .
    container_name: worker3
    hostname: worker3
    networks:
      - mpi_net

networks:
  mpi_net:

```

A.8 setup_ssh.sh

Configuração de SSH sem senha e criação do hostfile

```
#!/bin/bash
# Configura SSH sem senha entre os nos do cluster e cria o hostfile.
# Pode ser executado mais de uma vez sem causar problema.
set -e

echo "1/3 Gerando a chave no master..."
docker compose exec -T master bash -c '
mkdir -p /home/mpiuser/.ssh
if [ ! -f /home/mpiuser/.ssh/id_rsa ]; then
    ssh-keygen -t rsa -b 2048 -N "" -f /home/mpiuser/.ssh/id_rsa
fi
chown -R mpiuser:mpiuser /home/mpiuser/.ssh
chmod 700 /home/mpiuser/.ssh
'

# O -T evita que o docker anexe um terminal e insira caracteres de
# retorno de carro no meio da chave, o que a corromperia silenciosamente.
PUBKEY=$(docker compose exec -T master cat /home/mpiuser/.ssh/id_rsa.pub)

echo "2/3 Autorizando a chave em cada no..."
for node in master worker1 worker2 worker3; do
    echo "    -> $node"
    echo "$PUBKEY" | docker compose exec -T "$node" bash -c '
mkdir -p /home/mpiuser/.ssh
cat > /home/mpiuser/.ssh/authorized_keys
printf "Host *\n    StrictHostKeyChecking no\n    UserKnownHostsFile /dev/null\n" \
    > /home/mpiuser/.ssh/config
chown -R mpiuser:mpiuser /home/mpiuser/.ssh
chmod 700 /home/mpiuser/.ssh
chmod 600 /home/mpiuser/.ssh/authorized_keys /home/mpiuser/.ssh/config
'
done

echo "3/3 Criando o hostfile no master..."
docker compose exec -T master bash -c '
printf "master slots=2\nworker1 slots=2\nworker2 slots=2\nworker3 slots=2\n" \
    > /home/mpiuser/hosts
chown mpiuser:mpiuser /home/mpiuser/hosts
'

echo
echo "SSH configurado. Teste com:"
echo ' docker compose exec master su - mpiuser -c "mpirun --hostfile hosts --map-by node -np 4 hostname"'

```

A.9 run.sh

Distribuição do código aos nós e execução via mpirun

```
#!/bin/bash
# Copia um script para os 4 nos do cluster e executa via mpirun.
#
# Uso: ./run.sh exercicio1.py 600
#
# Sem o --map-by node o MPI colocaria 2 processos no master e 2 no
# worker1, deixando worker2 e worker3 parados, porque o hostfile
# declara slots=2 por maquina.

ARQUIVO=$1
N=$2

if [ -z "$ARQUIVO" ]; then
    echo "uso: ./run.sh <arquivo.py> [N]"
    exit 1
fi

for no in master worker1 worker2 worker3; do
    docker cp "$ARQUIVO" "$no":/home/mpiuser/ > /dev/null
done

docker compose exec master su - mpiuser -c \
    "mpirun --hostfile hosts --map-by node -np 4 python3 $ARQUIVO $N"

```

A.10 benchmark.sh

Execução das nove combinações e consolidação dos tempos

```
#!/bin/bash
# Roda as 3 versoes (sequencial, threads, MPI distribuido) para cada N
# e imprime a tabela de desempenho do Exercicio 1.
#
# Uso: ./benchmark.sh          -> roda N = 300, 600 e 1000
#      ./benchmark.sh 300 600 -> roda so os N informados
#
# Os resultados tambem sao gravados em resultados.txt.

set -u

NS=${@:-"300 600 1000"}
SAIDA="resultados.txt"

# Extrai o ultimo numero decimal da saida do programa. Funciona para os
# tres formatos de print porque em todos o tempo e o unico valor com
# casa decimal na linha.
extraí_ms() {
    grep -oE '[0-9]+\.[0-9]+' | tail -1
}

declare -A R

echo "Iniciando benchmark. Os N maiores demoram alguns minutos."
echo

for N in $NS; do
    echo "--- N = $N ---"

    echo -n " sequencial ... "
    t=$( { python3 sequencial.py "$N"; } 2>&1 | extraí_ms )
    R["$N,seq"]=${t:-ERRO}
    echo "${t:-ERRO} ms"

    echo -n " threads ..... "
    t=$( { python3 threads.py "$N"; } 2>&1 | extraí_ms )
    R["$N,thr"]=${t:-ERRO}
    echo "${t:-ERRO} ms"

    echo -n " MPI ..... "
    for no in master worker1 worker2 worker3; do
        docker cp exercicio1.py "$no":/home/mpiuser/ > /dev/null 2>&1
    done
    t=$( docker compose exec master su - mpiuser -c \
        "mpirun --hostfile hosts --map-by node -np 4 python3 exercicio1.py $N" \
        2>&1 | extraí_ms )
    R["$N,mpi"]=${t:-ERRO}
    echo "${t:-ERRO} ms"
    echo
done

# ---- tabela final -----
{
    echo
    echo "=====
    echo " Exercicio 1 - Multiplicacao de matrizes | tempos em ms"
    echo "=====
    printf " %-8s %14s %14s %14s\n" "N" "Sequencial" "Threads" "MPI (4 nos)"
    echo " -----
    for N in $NS; do
        printf " %-8s %14s %14s %14s\n" \
            "$N" "${R[$N,seq]}" "${R[$N,thr]}" "${R[$N,mpi]}"
    done
    echo "=====
    echo " Gerado em $(date '+%d/%m/%Y %H:%M') | nproc = $(nproc)"
    echo "=====
} | tee "$SAIDA"
```

```
echo  
echo "Tabela salva em $SAIDA"
```